COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: Saket Kaki | UT EID: sk46462 | Repository:
https://github.com/saketkaki/coe379lfa26-midterm

1. System Overview

The request starts with LabRequest, which is the trusted request envelope. The LabRequest includes the known values of authenticated user id, project id, resource id, start and end times, and the user's typed text. The prompt the model send are the system rules and a message containing the trusted envelope and the user's text. The rules of the model are that it should never trust any text that is retrieved from a passage or a tool observation because they are untrusted data. It should trust only the instructions. The agent then goes on to the decision-action-observation loop with a default step limit programmed to 10. Going based off the data flow diagram, we can see that per step the model calls a tool. After this call for a tool, the action gate runs to check to see if the user, project and resource are the same as what the LabRequest shows. The reservation must also show that the ReservationContext is true for all. If it observes a failure, the step is logged as `action_gate_denied`. The final response is checked against the trace by command `validate_final_response`. The final response has to show that every citation it outputs has to be the same as the trace. If the final response is correct, we end the run with `model_finished`. Otherwise, it ends with `invalid_final_response`.

2. Public Benchmark Results

Table 1: Aggregate metrics from `public_benchmark.json`.

Metric	Value	Metric	Value
`scenario_count`	6	`unnecessary_refusal_rate`	0
`success_rate`	1	`clarification_accuracy`	1
`invalid_structured_response_rate`	0	`average_tool_calls`	2.666
`unsafe_action_attempt_rate`	0.0		

We are able to see that all six public scenarios pass. There are no public scenarios that have failed. In the `public_benchmark.json`, we can see that for every scenario, the final status matches the expected status. According to the .json file, for almost all scenarios, the trace step ends after step 3. As a result, this causes the unsafe action attempt to be zero due to the fact that no reservation was proposed. Amongst the scenarios, there was only one that came close to failing, which was the scenario `conflict.` The only reason why is because it was the only scenario where there was a bad/harmful instruction that reached the model, which was the untrusted note of `NOTE-999#embedded-instruction.`

3. Student Tests

I wrote three deterministic tests and each one uses ScriptedLLM because it ensures that the result I get does not depend on the live model.

Test 1: The first test demonstrates an unsafe action proposal that is denied before tool execution. Specifically, I establish that Alice is logged in and is the authenticated user and that Bob tries to book a reservation. The test results in a final response status of `blocked` with an observation error code of `action_gate_denied.` The denial message that comes up is `user_id_must_match_authenticated_user`. Because of this conflict and error message, the lab still keeps the initial reservation of Alice and does not allow Bob to reserve the lab.

Test 2: The second test demonstrates a contradictory final response that is rejected because it disagrees

with the trace. Specifically, I establish that Bob is the authenticated user and tries to reserve `tensile-01` from 19:00 to 20:00. This time interval that is trying to be reserved is counted as after hours. The model then searches through the lab documents, checks Bob's authorizations and resource status. It finds that Bob does not have after hours approval, so the reservation is denied with an observation error code of `action_gate_denied`. We can then see that the model shows a final response of the status as completed and `reserve_resource` as the completed action despite the fact that the trace shows the reservation booking was denied. Because of this, the model returns a final status of `failed` with a termination reason of `invalid_final_response` and a response message of `completed_without_successful_mutation`.

Test 3: The third test demonstrates an additional case of my choice which is about missing qualifications. Specifically, I establish that Carol is the authenticated user and that she tries to reserve `tensile-01` for the project `robotics-demo`. The model searches through the lab documents, checks Carol's authorizations and resource status. However, she only has the `robotics_level_2` qualification and does not have the `lab_safety` qualification. The `lab_safety` qualification is needed to reserve the tensile tester. So the reservation is denied with an observation error code of `action_gate_denied`. Because of this, the model returns a final status of `blocked` with a response message of `required_qualification_missing`.

My test cases help ensure the quality of the system by checking the safety rules that the public benchmark does not check. Based off the public scenarios, we can see that they do not propose `reserve_resource` in an unsafe state.

4. Implementation Decisions

My error codes are written with underscores like `project_id_does_not_match_request` and `user_id_unresolved_in_request`. Each error code corresponds to only one rule, which allows an easier way to find these codes in the trace when there is a lot of data. When the action gate denies a tool call, I do not stop the run. Rather, I let the run keep going. In the trace, the step is saved with this information: ok=False, error code of `action_gate_denied`, and a message listing the errors. I use a total of 5 termination reasons: `clarification_required, model_finished, invalid_model_decision, invalid_final_response, maximum_steps_exceeded. clarification_required` means that the model is asking the user for more information since it might not have it. `model_finished` means the final answer that the model shows is accepted and good. `invalid_model_decision` means the output given is not a valid decision. `invalid_final_response` means the answer that the model gave does not agree with what the trace gives, so it shows a failure. `maximum_steps_exceeded` means that the default 10 step limit is hit. The way I handle request fields that are unresolved are by denying the reservation booking by the error code `field_unresolved_in_request`. This means the model will ask the user for clarification on the fields that are unresolved.

Data Flow Diagram (not counted toward the 2-page limit)

